

SEED Labs – Packet Sniffing and Spoofing

Joseph Du (jbdu@crimson.ua.edu)

CS438-001

Abstract— The purpose of this SEED lab is to gain an understanding of the practice of packet sniffing and packet spoofing, as well as learning how software tools for packet sniffing and packet spoofing are created. This is accomplished by implementing programs to perform packet sniffing and packet spoofing in both the Python and C programming languages.

I. INTRODUCTION AND LAB DEFINITION

In the age of wireless communications, more data is being sent over the open air than ever before. From a convenience standpoint, this technology has changed many things about everyday life, from the way people talk to each other, to shopping, banking, and doing business. From a security standpoint, however, sending potentially sensitive information through the open air poses a major risk of it landing in the wrong hands. This is where the techniques of packet sniffing and spoofing are important. While attackers may use these techniques to do harm, security experts must also understand them in order to protect against potential malicious activity.

There are many different software tools used to sniff and spoof packets. A popular one is Wireshark, which has been used in previous labs to study network protocols and movement of data through and across networks. This is done by monitoring a particular network interface on a given machine and capturing packets in a trace that holds the details of traffic across the interface. Looking at real network traffic can be helpful in understanding many networking concepts, particularly the layered architecture of protocols and the specifics of protocols themselves.

This lab, however, focuses on the implementation of tools like Wireshark, providing an explanation for the methods by which software tools can be used to monitor network traffic. This is achieved by completing exercises using the Python programming language and the “Scapy” library for it, which has functions to detect and record packets, as well as modify and customize packets to be sent. Additionally, the lab also uses a similar set of exercises to be completed in the C programming language using the “pcap” library, which contains structures and functions that enable us to capture traffic and spoof packets. Implementing these programs using these libraries gives hands-on experience in the technical aspects of the software and techniques used by attackers and defenders alike [1].

II. LAB SETUP

The steps for setting up this lab were particularly involved. Steps included downloading a virtual machine (VM) and configuring Docker containers to act as an attacker and two hosts, A and B [1]. While the lab documentation detailed several different ways to go about setting up this environment, I chose to download a pre-built SEED VM. This VM uses the Ubuntu distribution of Linux, specifically version 20.04 (also referred to as Focal Fossa) [2]. It also includes some command line aliasing to simplify Docker commands that will be used to configure the containers previously mentioned.

To run this VM on my machine, I first downloaded the VirtualBox software, which allows a user to upload a disk image for a particular VM and emulate it on their machine. In this case, the disk image was available for download on the SEED Security Labs website [3]. Once the VM was downloaded and running on my machine, it was time to configure the Docker containers that will be used to simulate a virtual wireless local area network (LAN). The resources for setting up the containers were provided by the SEED Labs documentation [2]. A Docker “YML” file contains all the information Docker needs to configure three containers in the desired configuration. Some examples of information include the names of each host, their IP addresses, their privilege levels, and information on the network and subnet the hosts reside on. Here, the aliases provided by the pre-built VM added convenience in building and starting the containers. The command, “docker-compose build” is aliased as “dcbuild,” and it builds the container image. The command, “docker-compose up” is aliased as “dcup,” and it starts the containers. The command “docker-compose down” is aliased as “dcdown,” and it shuts the containers down when necessary. Once the containers are up and running, there are two more aliases that are necessary for accessing them. The command, “dockps” is an alias that is used to print the IDs of the containers. The ID of a container is used in the command, “docksh” which is an alias that accesses a shell inside the container, enabling command execution [1].

Another important command is “ifconfig,” which displays information about network interfaces on Linux machines. This will be important to find the IP address of the VM, as well as the network ID created by Docker, which will be used in the programming exercises later in the lab [1]. For the attacker container, it is important to note that it was configured in host mode, meaning can see all of the host’s network interfaces. This

was necessary to enable packet sniffing because the containers are only able to see their own traffic by default [1].

III. TASK 1.1

Creating a packet sniffing program in Python using the Scapy library is as simple as a print statement and calling the “sniff” and “show” functions from the library. Python and Scapy were both preinstalled on the prebuilt VM, so no downloads were necessary. However, the Scapy library documentation was helpful in becoming familiar with the features and functions it offers [4]. A simple packet sniffing program is outlined in the lab directions. This program can be found in the additional files section of the lab submission labeled “task1.1.py.” The rest of this task will involve manipulating this program to satisfy certain requirements.

The first of these modifications has to do with the execution of the program. Packet sniffing requires root privilege, which is achieved by adding the “sudo” prefix before executing the program. The result of this is shown below.

```
[03/09/26]seed@VM:~/.../Labsetup$ sudo ./sniffer.py
###[ Ethernet ]###
  dst      = 52:55:0a:00:02:02
  src      = 08:00:27:50:20:ca
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 1479
  flags    = DF
  frag     = 0
  ttl      = 64
  proto    = icmp
  chksum   = 0x18c4
  src      = 10.0.2.15
  dst      = 8.8.8.8
  \options \
###[ ICMP ]###
  type     = echo-request
  code     = 0
  chksum   = 0x5d2a
  id       = 0x3
  seq      = 0x1
###[ Raw ]###
  load     = '\xafL\xaei\x00\x00\x00\x00H\r\x00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&\'()*+,-./01234567'
```

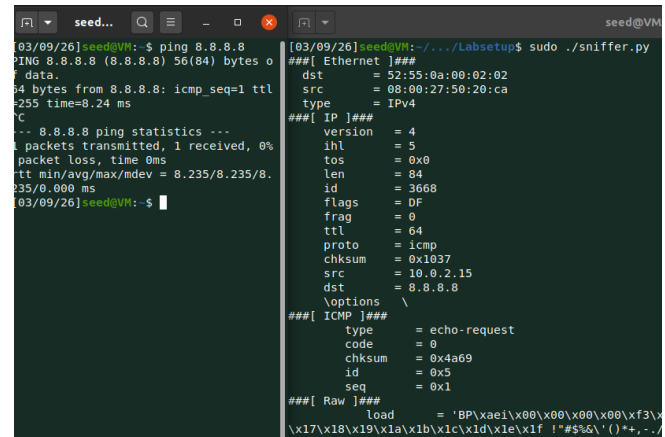
Fig. 1. A screenshot of the output of “task1.1.py,” displaying a packet that was sniffed using the Scapy library functions.

To demonstrate the necessity of root privilege to perform packet sniffing, an attempt to run the program from the “seed” user is shown below.

```
[03/09/26]seed@VM:~/.../Labsetup$ whoami
seed
[03/09/26]seed@VM:~/.../Labsetup$ sniffer.py
Traceback (most recent call last):
  File "./sniffer.py", line 7, in <module>
    pkt = sniff(iface=['br-f184ab6f6030', 'enp0s3'], filter='icmp', prn=
print_pkt)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line
1036, in sniff
    sniffer.run(*args, **kwargs)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line
894, in _run
    sniff_sockets.update(
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line
895, in <genexpr>
    (L2socket(type=ETH_P_ALL, iface=iface, *arg, **karg),
  File "/usr/local/lib/python3.8/dist-packages/scapy/arch/linux.py", lin
e 398, in _init_
    self.ins = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.h
tons(type)) # noqa: E501
  File "/usr/lib/python3.8/socket.py", line 231, in _init_
    socket.socket._init(self, family, type, proto, file)
PermissionError: [Errno 1] Operation not permitted
```

Fig. 2. A screenshot of the failed attempt to run the packet sniffer from the seed user.

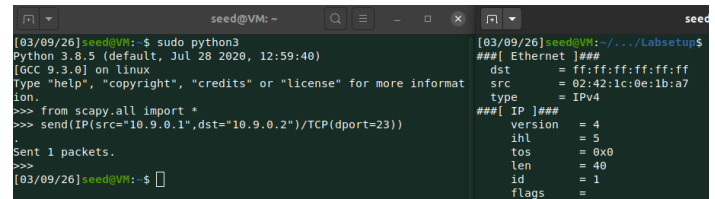
The next set of lab exercises involves capturing specific types of traffic. In order to do this, we must create filters that only pick up traffic if it satisfies a certain requirement. The syntax used in this step is called the Berkeley Packet Filter (BPF) syntax. This is helpful when sorting through a large amount of traffic and having a basic understanding of the topic will help to advance the functionality of our packet sniffer [5]. The first filter is already filled out in the sample code provided in the lab documentation. Using the expression, “filter=‘icmp’” in the program filters the traffic to only show ICMP packets. A sample execution of the working filter is shown below.



```
seed@VM:~/.../Labsetup$ sudo ./sniffer.py
[03/09/26]seed@VM:~/.../Labsetup$ sudo ./sniffer.py
###[ Ethernet ]###
  dst      = 52:55:0a:00:02:02
  src      = 08:00:27:50:20:ca
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 3668
  flags    = DF
  frag     = 0
  ttl      = 64
  proto    = icmp
  chksum   = 0x1037
  src      = 10.0.2.15
  dst      = 8.8.8.8
  \options \
###[ ICMP ]###
  type     = echo-request
  code     = 0
  chksum   = 0x4a69
  id       = 0x5
  seq      = 0x1
###[ Raw ]###
  load     = '\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&\'()*+,-./01234567'
```

Fig. 3. A screenshot of the sniffer program filtering for ICMP packets.

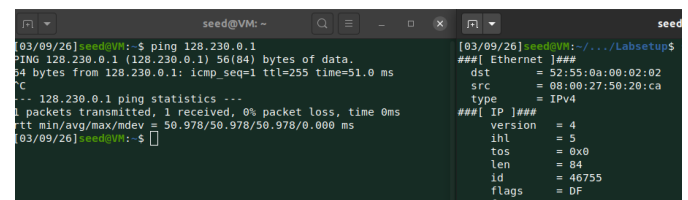
Moving to the next part of the exercise, the task is to capture a TCP packet with a certain source IP address and a destination port number of 23. This can be achieved by editing the filter parameter to include, “tcp and src 10.9.0.1 and dst port 23.” A demonstration of this filter can be found below.



```
seed@VM:~/.../Labsetup$ sudo python3
Python 3.8.5 (default, Jul 28 2020, 12:59:40)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more
>>> from scapy.all import *
>>> send(IP(src="10.9.0.1",dst="10.9.0.2")/TCP(dport=23))
Sent 1 packets.
>>>
[03/09/26]seed@VM:~/.../Labsetup$ sudo ./sniffer.py
###[ Ethernet ]###
  dst      = ff:ff:ff:ff:ff:ff
  src      = 02:42:1c:0e:1b:a7
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 40
  id       = 1
  flags    = 0
```

Fig. 4. A screenshot of the sniffer program with the filter configured for a specific source IP and destination port.

The last part of Task 1.1 involves capturing traffic from an arbitrary subnet. This can be represented in the filter parameter by adding “net” and appending the CIDR notation of the desired subnet, where the IP is followed by a slash and the subnet mask in decimal form. The example provided in the documentation is the subnet, “128.230.0.0/16.” This filter can be seen in action below.



```
seed@VM:~/.../Labsetup$ sudo ./sniffer.py
[03/09/26]seed@VM:~/.../Labsetup$ sudo ./sniffer.py
###[ Ethernet ]###
  dst      = 52:55:0a:00:02:02
  src      = 08:00:27:50:20:ca
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 84
  id       = 46755
  flags    = DF
  frag     = 0
```

Fig. 5. A screenshot of the sniffer program with the filter configured to detect traffic from a specified subnet.

IV. TASK 1.2

Now that the packet sniffing has been covered, it is time to analyze packet spoofing. Similar to the previous task, a spoofing program will be developed in Python using the Scapy library. The program to be used is provided in the lab documentation. Specifically, the spoofing performed here involves changing the IP address of an ICMP request packet and sending it from one container to another within the VM network. By prompting the other machine to reply to the request, we will know if we were successful in spoofing if we receive an ICMP reply from the target machine. In practice, this is achieved by running the program in a terminal, then using a tool like Wireshark to monitor the ICMP request and reply. As shown below, the program spoofed a packet with the destination IP “1.1.1.1” and received a reply from the machine.

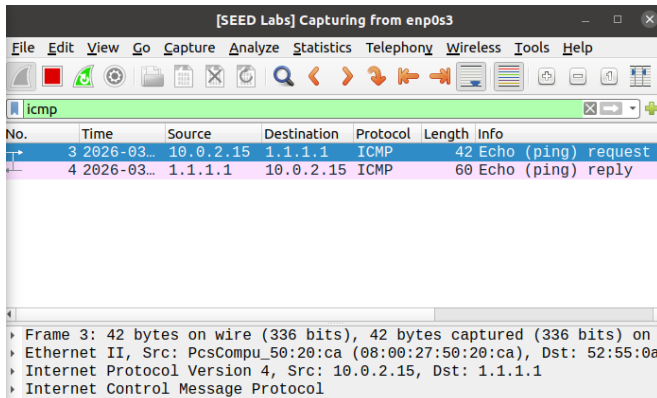


Fig. 6. A screenshot of Wireshark picking up spoofed packets from the spoofing program.

V. TASK 1.3

The next task is a departure from sniffing and spoofing, but focuses on the functionality of a different tool: traceroute. This tool is used to track the path data takes from the source to the destination [6]. This path may involve multiple routers with the number of hops between routers measured by the packet’s time to live (TTL), which will be manipulated in the program. Three iterations of this process are shown in the screenshot below.

```
[03/12/26]seed@VM:~/.../Labsetup$ sudo python3
Python 3.8.5 (default, Jul 28 2020, 12:59:40)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from scapy.all import *
>>> a=IP()
>>> a.dst='130.160.24.175'
>>> a.ttl=1
>>> b=ICMP()
>>> send(a/b)
.
Sent 1 packets.
>>> a.ttl=2
>>> send(a/b)
.
Sent 1 packets.
>>> a.ttl=3
>>> send(a/b)
.
Sent 1 packets.
>>>
[03/12/26]seed@VM:~/.../Labsetup$
```

Fig. 7. A screenshot of the simulation of the traceroute tool by changing the packet TTL in each iteration.

An interesting issue arose in completing this task. In theory, incrementing the TTL by 1 should reveal each router along the path. However, in reality, the data only crossed one node when it left the VM. Analyzing traffic on the VM does not account for the traffic on the machine hosting the VM, therefore this traceroute simulation did not necessarily reveal a path of routers the data took from the source to the destination. The results are pictured in Wireshark, demonstrating the seemingly direct connection between the VM and the server [7].

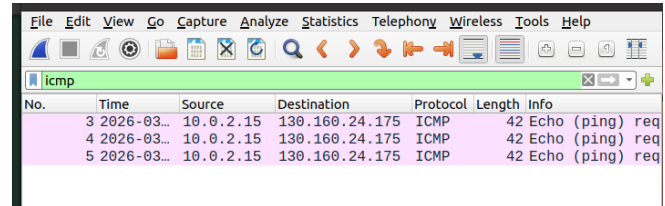


Fig. 8. A screenshot of the traffic picked up by Wireshark.

VI. TASK 1.4

Returning to the topic of packet sniffing and spoofing, the objective of this task is to create a program that can perform both actions. This Python/Scapy program will detect ICMP requests and spoof ICMP requests using the requested IP address. This will allow us to impersonate a different machine using its IP address, sending ICMP replies regardless of whether the destination is alive or not. The first test case for this program provided in the lab documentation is an IP address of a nonexistent host on the internet, 1.2.3.4. Because the host does not exist, we should not expect to get a reply. However, our sniffing and spoofing program detects the activity and sends a reply from the spoofed IP. The output of the program using this test case is shown below.

```
seed@VM: ~/.../Labsetup
[03/12/26]seed@VM:~/.../Labsetup$ ping -c 5 1.2.3.4
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data:
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=20.4 ms
64 bytes from 1.2.3.4: icmp_seq=2 ttl=64 time=26.0 ms
64 bytes from 1.2.3.4: icmp_seq=3 ttl=64 time=23.9 ms
64 bytes from 1.2.3.4: icmp_seq=4 ttl=64 time=22.1 ms
64 bytes from 1.2.3.4: icmp_seq=5 ttl=64 time=28.7 ms

--- 1.2.3.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4016ms
rtt min/avg/max/mdev = 20.434/24.219/28.653/2.880 ms
[03/12/26]seed@VM:~/.../Labsetup$
```

Fig. 9. A screenshot of the combined sniffing and spoofing program to the destination IP, 1.2.3.4.

The second test case is the IP of a nonexistent host within the same LAN as the machine running the program. This IP address is 10.9.0.99. The packet sniffer and spoofer does not send spoofed replies from this address. Even though this is contrary to the description in the beginning, it is expected behavior in the network. The expected result, a “destination host unreachable” error message, is shown below.

```

seed@VM: ~/.../Labsetup
[03/12/26]seed@VM:~/.../Labsetup$ ping -c 5 10.9.0.99
PING 10.9.0.99 (10.9.0.99) 56(84) bytes of data.
From 10.9.0.1 icmp_seq=1 Destination Host Unreachable
From 10.9.0.1 icmp_seq=2 Destination Host Unreachable
From 10.9.0.1 icmp_seq=3 Destination Host Unreachable
From 10.9.0.1 icmp_seq=4 Destination Host Unreachable
From 10.9.0.1 icmp_seq=5 Destination Host Unreachable

--- 10.9.0.99 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4087ms
pipe 3
[03/12/26]seed@VM:~/.../Labsetup$

```

Fig. 10. A screenshot of the program to the destination IP, 10.9.0.99.

The reason this behavior is expected has to do with the ARP protocol. Address resolution protocol (ARP) is used to resolve IP addresses to MAC addresses in order to send data from source to destination. Since this IP is on the same LAN, the host attempts to use ARP to resolve it to a device's MAC address. However, this device does not actually exist. Therefore, there is no ARP reply, and therefore no ICMP request is sent at all. This explains why the sniffer and spoofer program does not spoof ICMP replies from this IP address. In the following screenshot, the difference between the IPs and their routes is shown using the "IP route get" command.

```

seed@VM: ~/.../Labsetup
[03/12/26]seed@VM:~/.../Labsetup$ ip route get 1.2.3.4
1.2.3.4 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000
cache
[03/12/26]seed@VM:~/.../Labsetup$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000
cache
[03/12/26]seed@VM:~/.../Labsetup$ ip route get 10.9.0.99
10.9.0.99 dev br-d00134b4fe17 src 10.9.0.1 uid 1000
cache
[03/12/26]seed@VM:~/.../Labsetup$

```

Fig. 11. A screenshot of the output of "ip route get" for each of the three test IP addresses.

The difference in this IP on the LAN is it lacks a router IP address like the other IPs. Therefore, packets attempt to travel directly to the destination rather than to an intermediate router.

Finally, the last test case given in the lab documentation is to ping an existing host on the internet with the IP address 8.8.8.8. This results in actual ICMP replies from the server, as well as spoofed replies fabricated by the program. Therefore, there are more replies than requests and some are marked as duplicates in the output of the ping command, as shown below.

```

seed@VM: ~/.../Labsetup
[03/12/26]seed@VM:~/.../Labsetup$ ping -c 5 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=8.88 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=22.4 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=6.43 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=9.32 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=15.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=18.0 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=8.45 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=26.8 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=5 ttl=64 time=16.4 ms

--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, +4 duplicates, 0% packet loss, time 4013ms
rtt min/avg/max/mdev = 6.427/14.673/26.778/6.576 ms
[03/12/26]seed@VM:~/.../Labsetup$

```

Fig. 12. A screenshot of the program to the destination IP, 8.8.8.8.

This is one warning sign to look out for when analyzing network traffic. If there is a suspicious amount of duplicate

ICMP packets, it could indicate that an attacker is sniffing and spoofing packets on that network.

VII. TASK 2.1

The next set of tasks focuses on the same principles of packet sniffing and spoofing, but the implementation will be in the C programming language using the "pcap" library. For this task, sample code is provided for implementing a packet sniffer in C [8]. This program is being submitted in the other files in the file titled, "task2.1.c."

The first question in the lab documentation asks about the sequence of library calls that are used in sniffer programs. A typical execution of a packet sniffing program must first open a session on the network interface. The library function responsible for this is "pcap_open_live." This function takes the device name, buffer size, promiscuous mode flag, and timeout as parameters in order to initialize a session. Next, the filter must be applied to the traffic being picked up. This is done using the library functions, "pcap_compile" and "pcap_setfilter." Together, these functions convert the human-readable filter into kernel instructions, then apply the filter to the traffic being captured. Next, the program loops continuously while it captures packets. The library function used for this is "pcap_loop." In this implementation, a "-1" in the parameters means the program will capture packets indefinitely until it is terminated. Finally, the session must be closed using "pcap_close." This ensures the session is closed and resources are released properly.

The next question asks why root privilege is required to run packet sniffer programs. Root privilege is required because packet sniffing programs need direct access to the network interface card. Since a part of the hardware needs to be accessed directly, the operating system requires the program to have root privilege to perform kernel-level operations. Without root privilege, the program would fail at the call to "pcap_open_live," since the function attempts to open the network interface.

The last part of this task involves toggling promiscuous mode on and off to observe the differences. When promiscuous mode is enabled with a value of 1, the network interface picks up all traffic on the network segment. An example of this is shown below, where host A pings host B and the attacker host picks up the traffic.

```

seed@VM: ~/.../Labsetup
[03/13/26]seed@VM:~/.../Labsetup$ docker exec -it hostA-10.9.0.5 bash
root@VM:/# cd tmp
root@VM:tmp# ./sniff
[03/13/26]seed@VM:~/.../Labsetup$ docksh 66f
root@95be42d59f22:/# ping -c 3 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.103 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.113 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.119 ms

--- 10.9.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 0.103/0.111/0.119/0.006 ms
root@95be42d59f22:/#

```

Fig. 13. A screenshot of the program running in promiscuous mode.

Alternatively, turning promiscuous mode off with a value of 0 causes the network interface to pick up only the traffic that is meant for it. An example of this is shown below with host A pinging host B, and the attacker host does not pick up the traffic.

```

[03/13/26]seed@VM:~/... [03/13/26]seed@VM:~$ docker exec -it hostA-10.9.0.5 ping -c 3 10.9.0.6
root@VM:/# cd tmp
root@VM:/tmp# ./sniff
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.138 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.112 ms

--- 10.9.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 205ms
rtt min/avg/max/mdev = 0.053/0.101/0.138/0.035 ms
[03/13/26]seed@VM:~$

```

Fig. 14. A screenshot of the program running with promiscuous mode turned off.

The next part of this task involves creating filters for the sniffer. The first filter will capture ICMP packets between two specific hosts. This is achieved by setting the filter expression equal to “icmp and host 10.9.0.5 and host 10.9.0.6.” These IP addresses represent the containers, host A and host B, so the attacker host will detect ICMP packets sent between the two. A demonstration of this filter in action can be found in the screenshot below.

```

[03/13/26]seed@VM:~/... [03/13/26]seed@VM:~$ docker exec -it hostA-10.9.0.5 ping -c 3 10.9.0.6
root@VM:/# cd tmp
root@VM:/tmp# ./sniff
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.174 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.116 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.055 ms

--- 10.9.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 205ms
rtt min/avg/max/mdev = 0.055/0.115/0.174/0.048 ms
[03/13/26]seed@VM:~$

```

Fig. 15. A screenshot of the program running with the filter configured for ICMP packets between two hosts.

The other filter to be created should capture TCP packets that are destined for port numbers 10 to 100. The filter to accomplish this task is “tcp dst portrange 10-100.” The functionality of this filter is displayed in the screenshot below, using the netcat tool to generate the traffic.

```

[03/13/26]seed@VM:~/... root@95be42d59f22:/# nc 10.9.0.6 50
root@VM:/# cd tmp
root@VM:/tmp# ./sniff
Got a packet

```

Fig. 16. A screenshot of the program running with the filter set for traffic on ports 10-100.

The last part of this task involves sniffing passwords. This can be done by sniffing Telnet packets since this method of communication is unencrypted. Therefore, credentials are transported in plaintext and can be picked up by a basic packet sniffer. To accomplish this, the filter must be set to “tcp port 23,” which corresponds to Telnet. The program was modified to print the data it picks up so the plaintext password can be sniffed. A demonstration of this is shown below, with the password “dees” highlighted in the capture with a yellow box.

```

[03/13/26]seed@VM:~/... 794403cd68ea login: seed
TCP Payload (10 bytes): Password:
Got a packet
Got a packet
TCP Payload (1 bytes): d
Got a packet
TCP Payload (1 bytes): e
Got a packet
TCP Payload (1 bytes): e
Got a packet
TCP Payload (1 bytes): s

```

Fig. 17. A screenshot of the plaintext password being picked up by the sniffer program.

VIII. TASK 2.2

Now that we have programmed a packet sniffer, it is time to program a packet spoofer. Part of the program is provided in the lab documentation, specifically the part that deals with raw sockets. Raw sockets, along with root privilege, allow users to change the field values of IP packets, enabling packet spoofing [1]. A demonstration of the packet spoofing program is shown below, with the commands in the terminal visible and the spoofed packets captured by Wireshark.

```

[03/13/26]seed@VM:~/.../Labsetup$ sudo ./spoofer 10.9.0.5 10.9.0.6
Spoofed ICMP echo request sent from 10.9.0.5 to 10.9.0.6
[03/13/26]seed@VM:~/.../Labsetup$

```

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-03-...	10.9.0.5	10.9.0.6	ICMP	62	Echo (ping) request
2	2026-03-...	10.9.0.5	10.9.0.6	ICMP	62	Echo (ping) request
3	2026-03-...	10.9.0.6	10.9.0.5	ICMP	62	Echo (ping) reply
4	2026-03-...	10.9.0.6	10.9.0.5	ICMP	62	Echo (ping) reply

Fig. 18. A screenshot of the spoofer program execution and the spoofed traffic recorded in Wireshark.

The next step is to spoof ICMP packets to a remote server on the internet. The server, if it is alive, should send an ICMP reply to the spoofed IP address if the program is functional. In the screenshot below, the terminal commands are shown that use the program to send spoofed packets, then the reply from the remote server is captured in Wireshark, proving the functionality of the program.

```

[03/13/26]seed@VM:~/.../Labsetup$ sudo ./spoofer 10.9.0.5 8.8.8.8
Spoofed ICMP echo request sent from 10.9.0.5 to 8.8.8.8
[03/13/26]seed@VM:~/.../Labsetup$

```

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-03-...	10.0.2.15	8.8.8.8	ICMP	62	Echo (ping) request
2	2026-03-...	8.8.8.8	10.0.2.15	ICMP	62	Echo (ping) reply
3	2026-03-...	8.8.8.8	10.9.0.5	ICMP	62	Echo (ping) reply
4	2026-03-...	8.8.8.8	10.9.0.5	ICMP	62	Echo (ping) reply

Fig. 19. A screenshot of the spoofer program execution and the spoofed traffic from a remote server recorded in Wireshark.

The first question asks if the spoofing program can set the packet length to any arbitrary value regardless of the size of the actual packet. In theory, the “ip_len” field can be controlled by the user and set to any value, regardless of the actual size. This can be used to overwhelm or confuse a host by making a packet seem larger than it actually is. However, modern network infrastructure like routers or firewalls may reject suspicious packets if this field does not match the actual length.

The next question asks if the checksum of the IP header must be calculated when using raw socket programming. Calculating the checksum is good practice and may be necessary for certain

applications of the packet spoofer program. For example, if a packet with an incorrect checksum is sent over a router, it will be detected as an error and the packet may be discarded. This would prevent the spoofed packets from being received.

The last question asks why root privilege is required to use raw sockets in programs. This has to do with the kernel, which restricts access to raw sockets. The reason for this restriction is to prevent any user from creating packets with arbitrary headers. Without root privilege, the program would fail at the “socket” function call.

IX. TASK 2.3

The final task involves combining the functionality of the packet sniffer and packet spoofer. Similar to the example in Python, this version in C will listen for any ICMP request packets sent from a host, then spoof ICMP responses and send them to the host. This will make it look like the destination host is alive to the sender, regardless of its actual status. In the example below, host A pings a nonexistent host on the LAN at the IP address, “192.0.2.1.” This private IP address is reserved for testing purposes, so a reply to the ping is not expected. However, the sniff and spoof program will generate a spoofed reply, giving the impression that there is a host at that address.

Fig. 20. A screenshot of the sniffer and spoofer program spoofing ICMP replies from a nonexistent LAN host.

X. CONCLUSION

This lab provided hands-on experience with the fundamental concepts of packet sniffing and packet spoofing. This reinforced how network traffic can be monitored, analyzed, and manipulated. The lab demonstrated how these techniques operate through implementation in both Python using Scapy and C using the pcap library and raw sockets. The exercises provided a deeper look into the creation of network tools and highlighted the importance of understanding them for both attackers and defenders.

Important security concepts were discussed, including root privilege requirements, promiscuous mode, ARP behavior, and ICMP packet handling. Observing the differences between real and spoofed traffic using tools like ping and Wireshark helped to clarify how to detect malicious activity. Knowing the signs of malicious activity is essential to protecting a network from attacks.

In conclusion, this lab strengthened practical networking and security skills through implementation of packet sniffing and spoofing programs. Understanding these techniques is critical for designing secure systems and recognizing potential attacks.

REFERENCES

- [1] W. Du, “SEED Labs – Packet Sniffing and Spoofing Lab,” SEED Security Labs, 2020.
- [2] W. Du, “Packet Sniffing and Spoofing Lab,” SEED Labs. [Online]. Available:

https://seedsecuritylabs.org/Labs_20.04/Networking/Sniffing_Spoofing/. Accessed: Mar. 18, 2026.

- [3] W. Du, “Lab Setup,” SEED Labs. [Online]. Available: <https://seedsecuritylabs.org/labsetup.html>. Accessed: Mar. 18, 2026
- [4] P. Biondi, “Scapy: Packet Manipulation Tool,” [Online]. Available: <https://scapy.net/>. Accessed: Mar. 18, 2026.
- [5] IBM, “Berkeley Packet Filters,” IBM Documentation, [Online]. Available: <https://www.ibm.com/docs/en/qsis/7.4.0?topic=queries-berkeley-packet-filters>. Accessed: Mar. 18, 2026.
- [6] Fortinet, “What Is Traceroute?,” Fortinet Cybersecurity Glossary, [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/traceroutes>. Accessed: Mar. 18, 2026.
- [7] Superuser, “Traceroute google.com returns only one hop on Kali VM,” Superuser, [Online]. Available: <https://superuser.com/questions/1930224/traceroute-google-com-returns-only-one-hop-on-kali-vm>. Accessed: Mar. 18, 2026.
- [8] Tcpdump Group, “libpcap,” [Online]. Available: <https://www.tcpdump.org/pcap.html>. Accessed: Mar. 18, 2026.